50.007 Machine Learning

Neural Networks

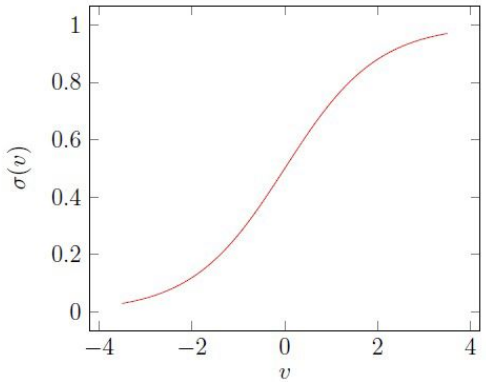
Roy Ka-Wei Lee
Assistant Professor, ISTD, SUTD

SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

Recap: Logistics Regression

Hypothesis function

$$h(\theta) = \frac{1}{1+e^{-\theta^T x}}$$



Training with the Cost Function

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

$\}$

Prediction

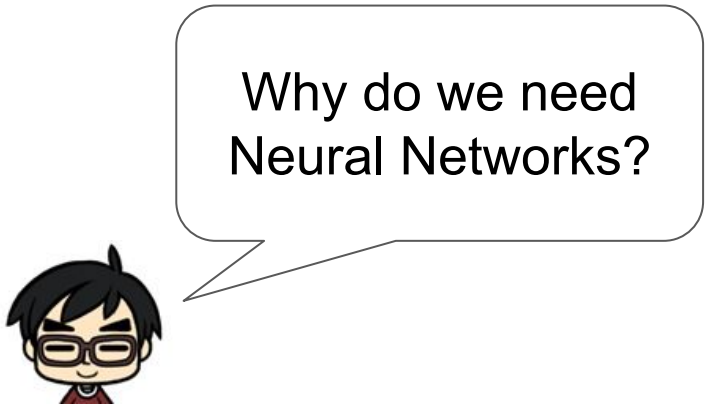
$$log\frac{p(y=1|x)}{p(y=0|x)} = \theta^T x + \theta_0 > 0?$$

If yes: positive, otherwise negative!

Learning Objectives

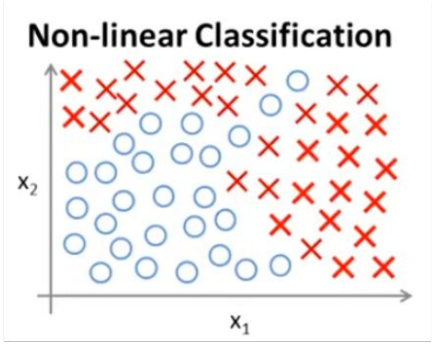
You should be able to know:

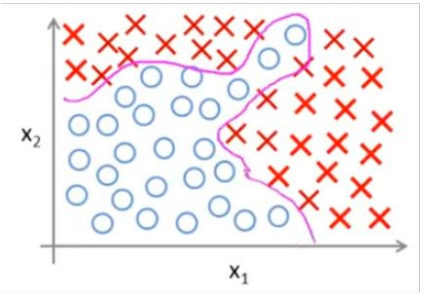
- Why do we need Neural Network?
- What is Neural Network (NN)? Explain the topology of a NN.
- Why can Neural Networks work?
- How to train Neural Network using backpropagation?


Why do we need
Neural Networks?

From logistic regression to Neural Networks

Consider the following binary classification problem:

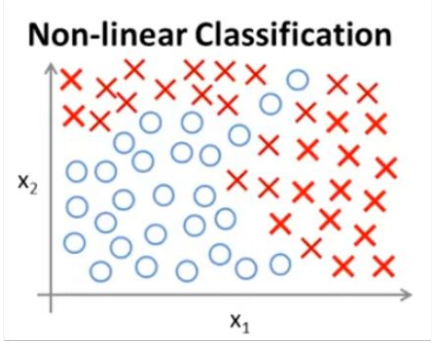


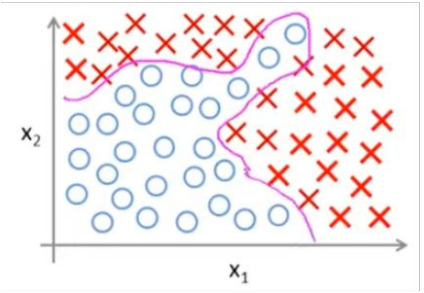


Can we find a hypothesis of logistic regression that can separate them?

From logistic regression to Neural Networks

Consider the following binary classification problem:





If we add ***a lot of polynomial terms***, we may find one…

$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \cdots..)$$

$$g(z) = \frac{1}{1+e^{-z}}$$ ⟶ will make the result from 0~1.

This works well when you have **TWO** features, because you can include all those polynomial terms of x_1 and x_2.

However, what happens if we have more features, not only x_1 and x_2?

For example: Housing classification problem

You want to predict one of the odds that a house would be sold within the 6 months.

We may have hundreds of features:

x_1 = size,
x_2 = age,
x_3= number of floors,
x_4= number of rooms,
...
x_{100}

For example: Housing classification problem

You want to predict one of the odds that a house would be sold within the 6 months.

We may have hundreds of features:

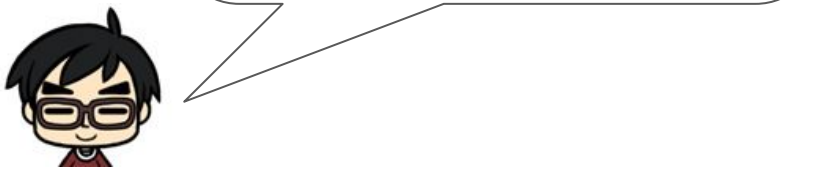
x_1 = size,
x_2 = age,
x_3 = number of floors,
x_4 = number of rooms,
...
x_{100}

+

$x_1^2, x_1x_2, x_1x_3, \ldots\ldots$
$x_2^2, x_2x_3, \ldots\ldots$
.....
$x_{100}^2, \ldots$

If you include 2nd order polynomial terms, there will have about **5000** features for ***n*=100**...
If you also include 3rd order polynomials, you may end up with **~170,000** features.
These added terms might end up overfitting, and they are computationally expensive. !

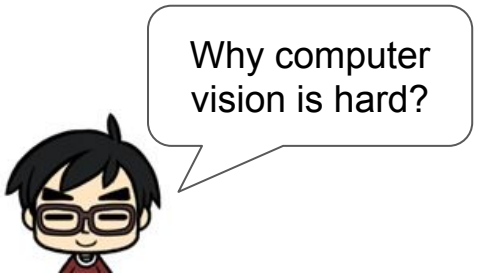
Thus, adding a lot of polynomial terms in logistic regression doesn't seem a good way when n is large...


Unfortunately, in many machine learning applications, n is very large… For example, computer vision…

A computer vision example - Car Recognition

You want to use ML to train a classifier to recognize whether an image is a car.

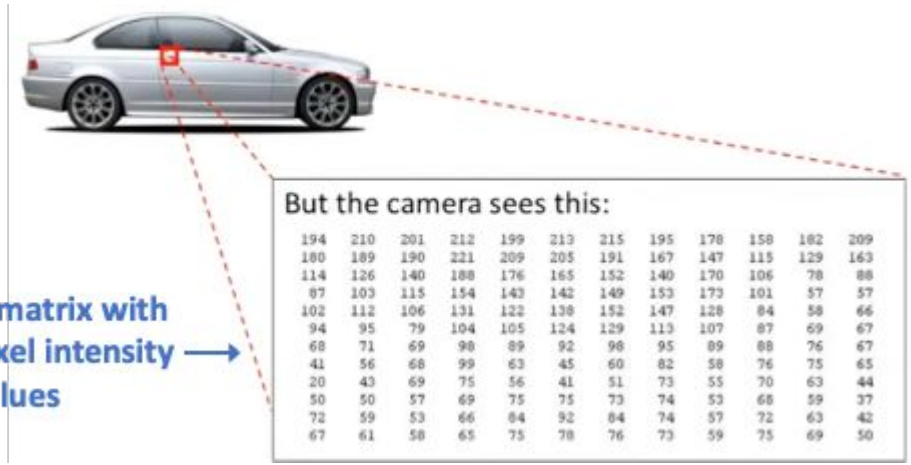
What you see is a car:

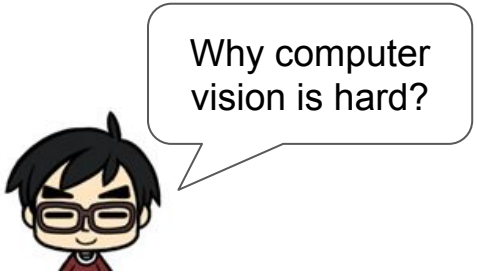


A computer vision example - Car Recognition

You want to use ML to train a classifier to recognize whether an image is a car.

What you see is a car:



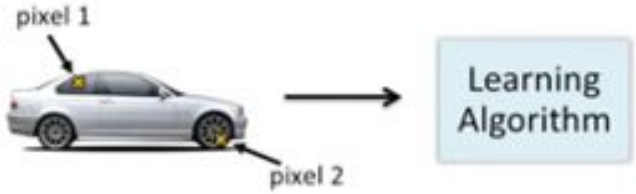


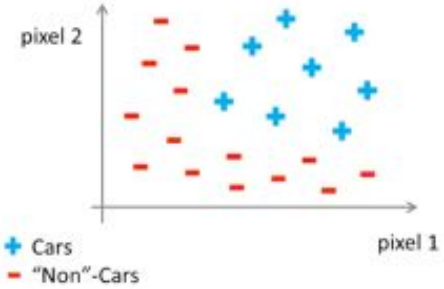
Computer vision aims to understand this is a door handle of the car by looking at the matrix of intensity values.

A computer vision example - Car Recognition

You want to use ML to train a classifier to recognize whether an image is a car.

We need a nonlinear hypothesis that separates 2 classes.





Suppose the image is grey and its resolution is 50*50 → n = 2500 (7500 if RGB)

$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

If we want to learn a non-linear hypothesis function with quadratic features such as $x_1 x_2$, we may end up with **3 million features**!

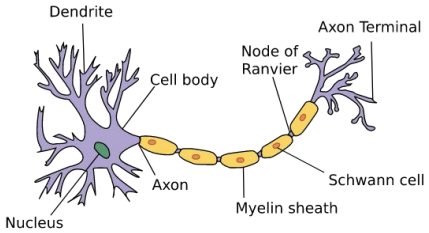
From logistic regression to Neural Networks

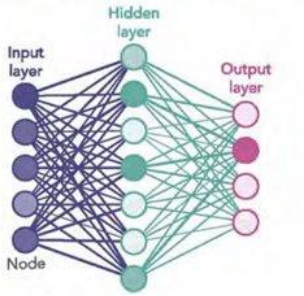
- **Logistic regression** together with adding quadratic and cubic features, is not a good way to learn complex nonlinear hypothesis, especially when n is large, since we end up with too many features.
- **Neural Networks** is a much better way to learn complex nonlinear hypothesis even when n is large, because it **learns from data**.

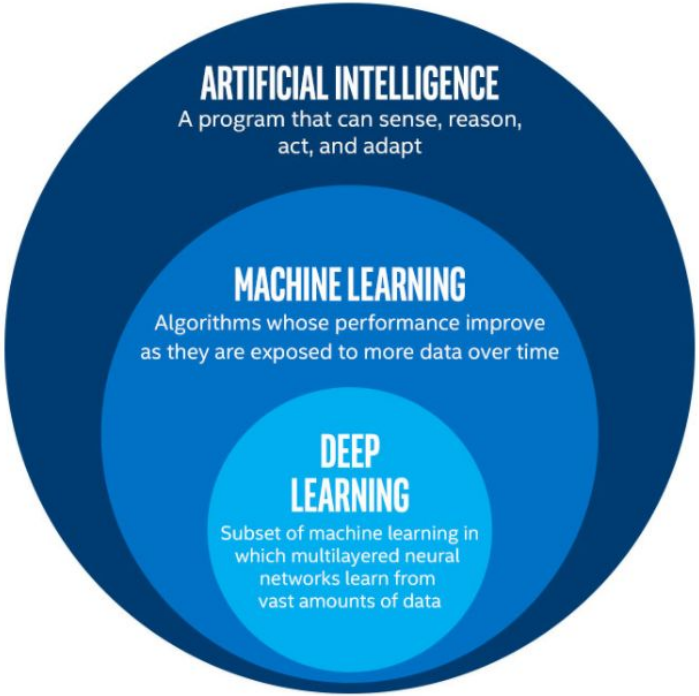
Big Picture of Neural Networks

What is a Neural Network?

- Neural networks are a **subset** of machine learning and are at the heart of deep learning algorithms. Their name and structure are inspired by the human brain, mimicking the way that biological neurons signal to one another.







What is a Neural Network?

- Neural networks rely on training data to learn and improve their accuracy over time.

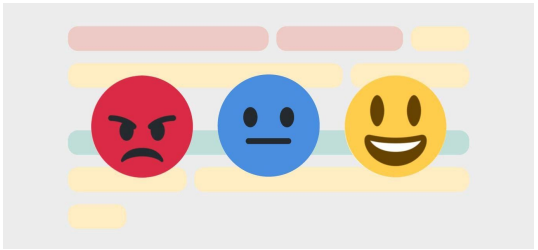
- Once these learning algorithms are trained, they are powerful tools in computer science and artificial intelligence, allowing us to classify and cluster data at a high velocity.

Types of Neural Network

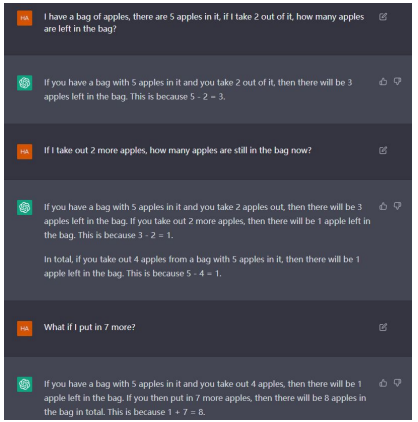
- Neural networks can be classified into different types, which are used for **different purposes.** The most common types of neural networks that you'll come across for its common use cases:
  - Deep Neural Networks (DNNs)
  - Convolutional neural networks (CNNs) [Week 3]
  - Recurrent neural networks (RNNs) [Week 4]

Applications of Neural Network

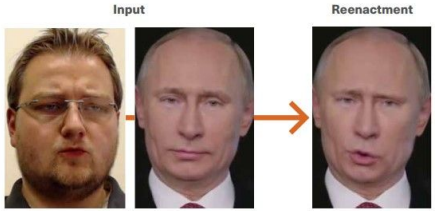
Facial Recognition



Sentiment Analysis



Dialogue systems (ChatGPT)



Video Generation (DeepFake)

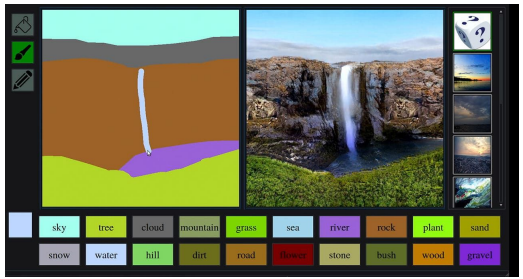


Image Generation

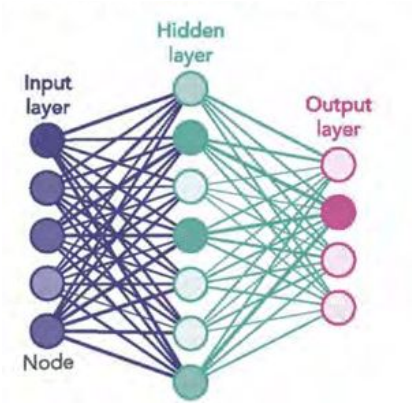
Many many more…

Cons of Neural Network

- “Black box" nature - their mechanisms of making decisions are not explicitly accessible to human cognition.
- Greater computational burden & slow implementation (sustainability issues)
- Proneness to overfitting/underfitting
- High dependance on training data - poor quality data makes bad models

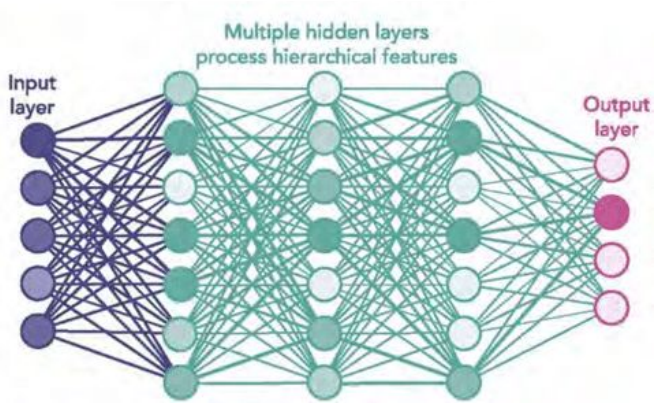
Shallow vs Deep Neural Network

- A **shallow** neural network has three layers of neurons that process inputs and generate outputs.



Shallow vs Deep Neural Network

- A **Deep** Neural Network (DNN) has **two or more** “hidden layers” of neurons that process inputs.



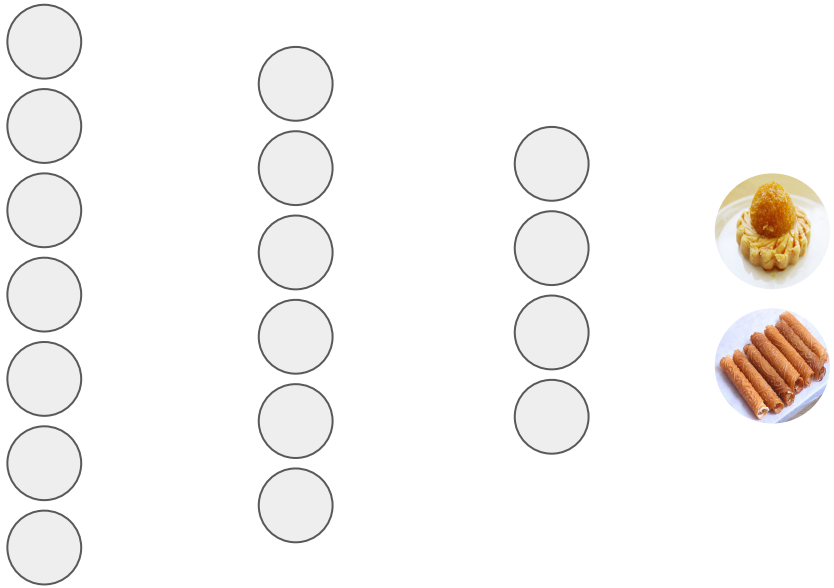
While shallow neural networks can tackle equally complex problems, deep learning networks are more accurate and **improve in accuracy as more neuron layers are added**.

High-Level Concept of Neural Network

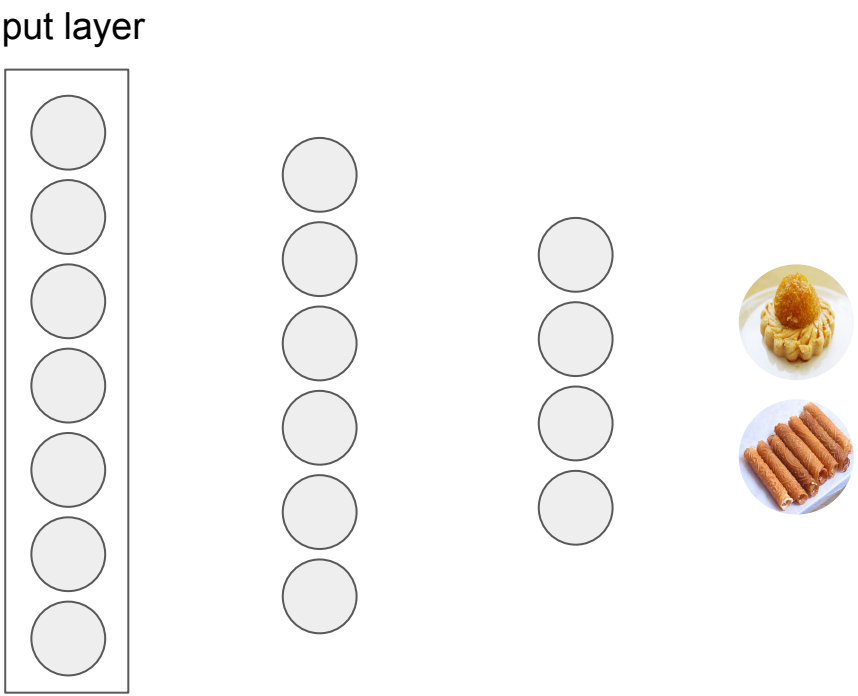
- Task: Classify the image - Pineapple Tart or Love Letter

High-Level Concept of Neural Network

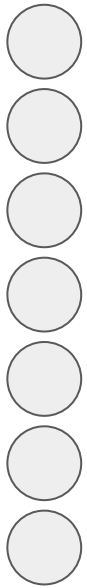
- Construct the neural network which makes up of neurons

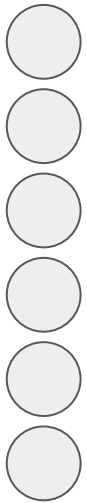


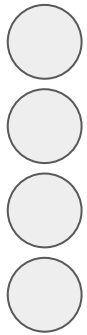
High-Level Concept of Neural Network



High-Level Concept of Neural Network

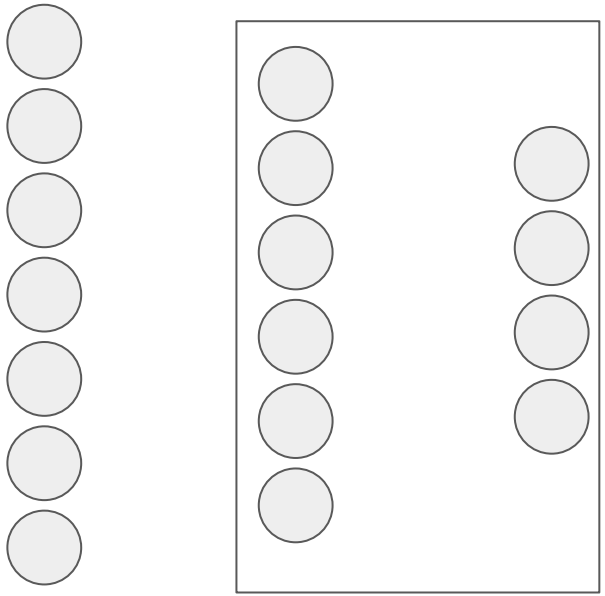






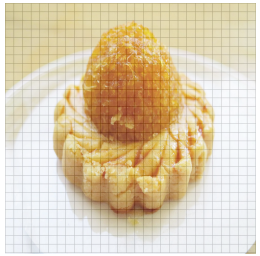
Output layer

High-Level Concept of Neural Network



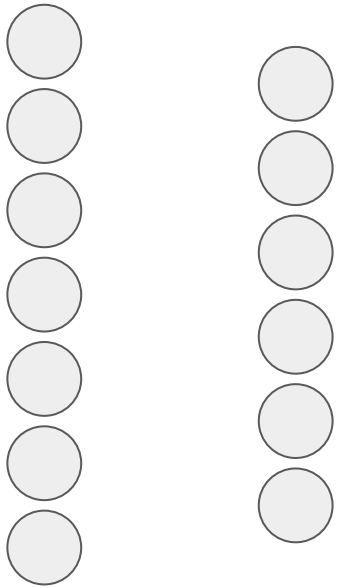
High-Level Concept of Neural Network

28

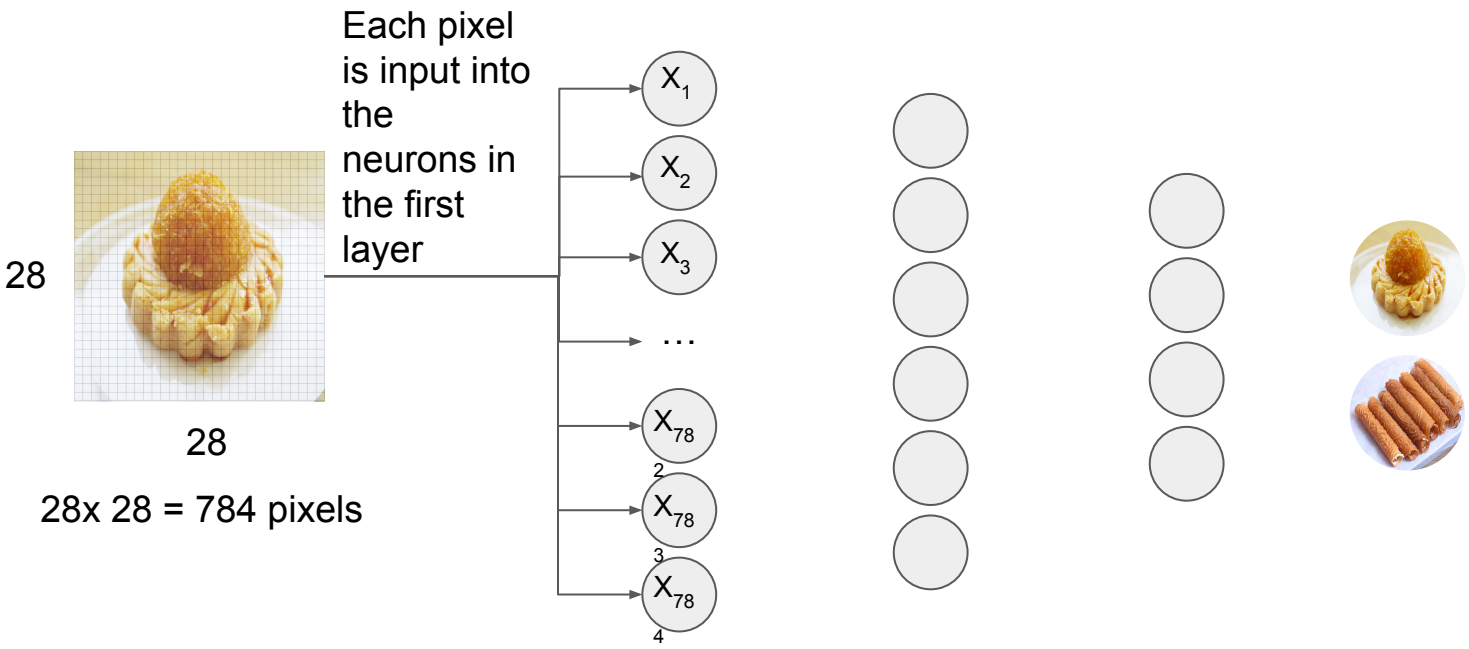


28

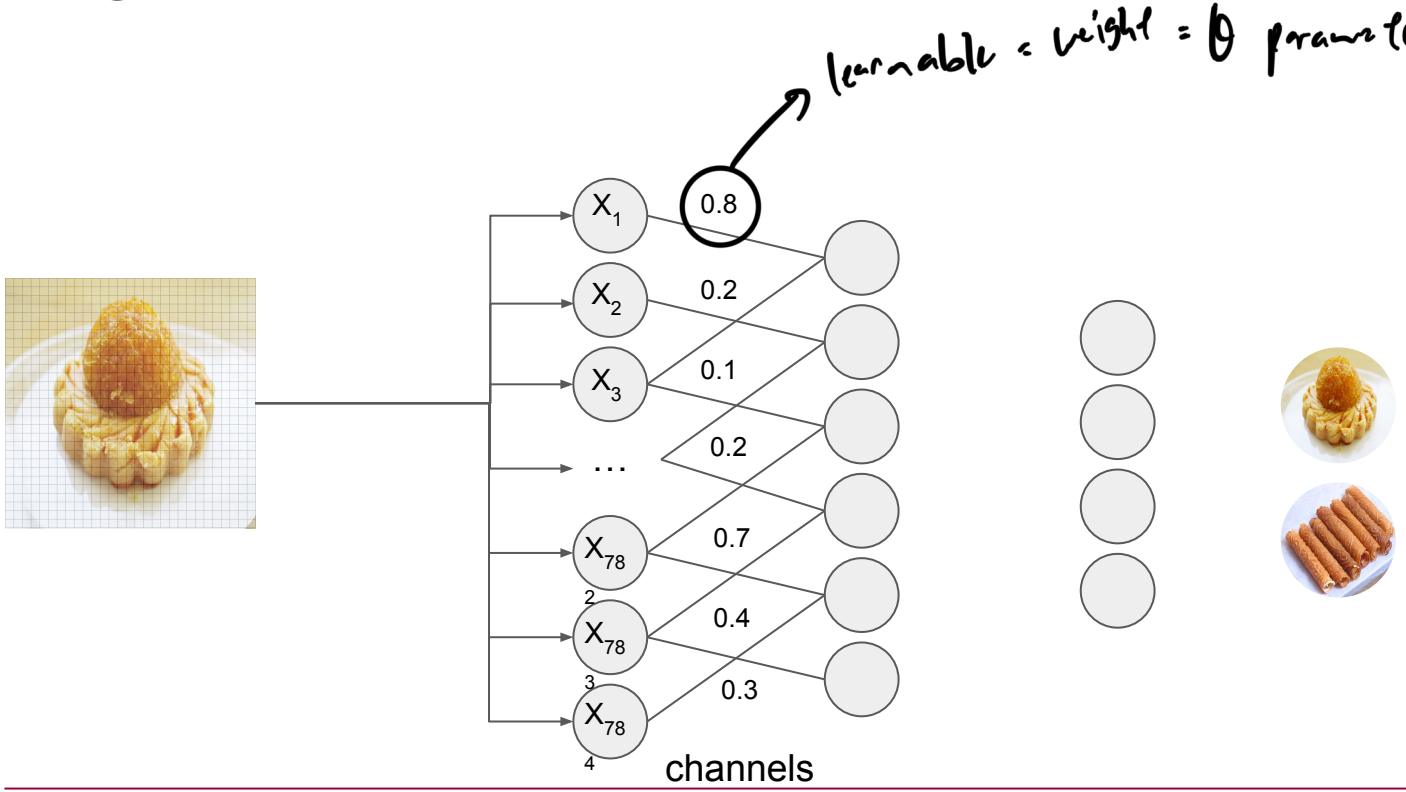
28x 28 = 784 pixels



High-Level Concept of Neural Network

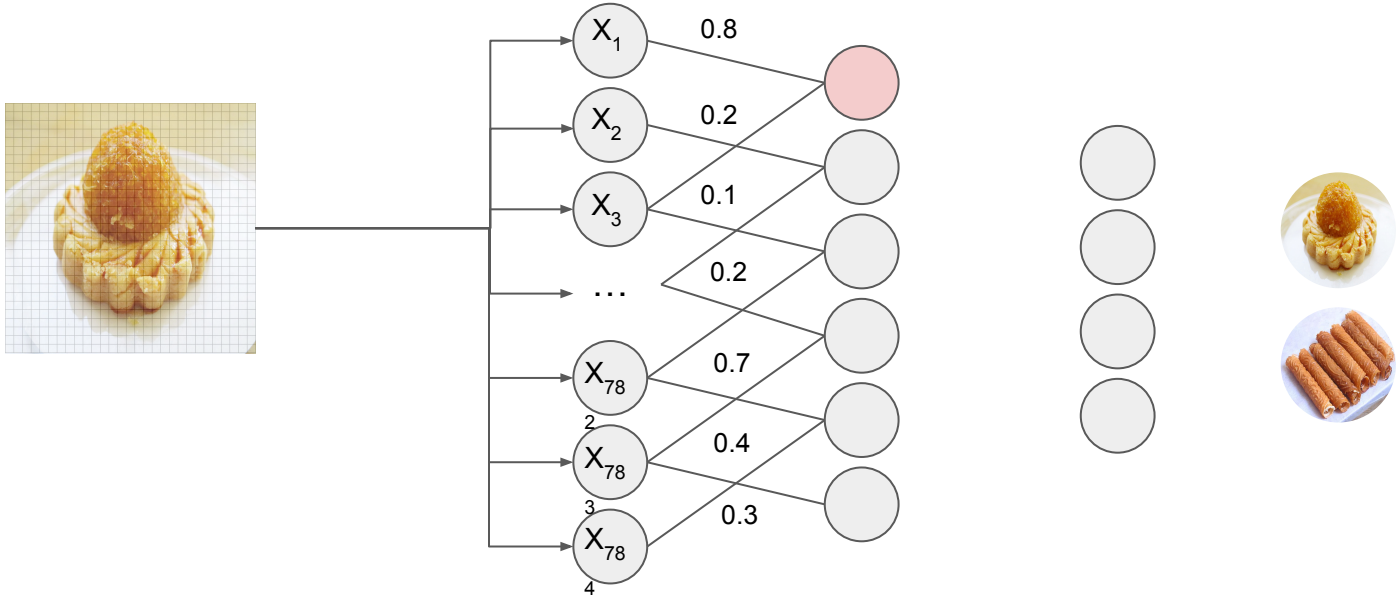


High-Level Concept of Neural Network



High-Level Concept of Neural Network

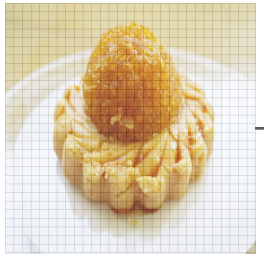
Input to neuron in hidden layer = (x_1*0.8 + x_3* 0.2)

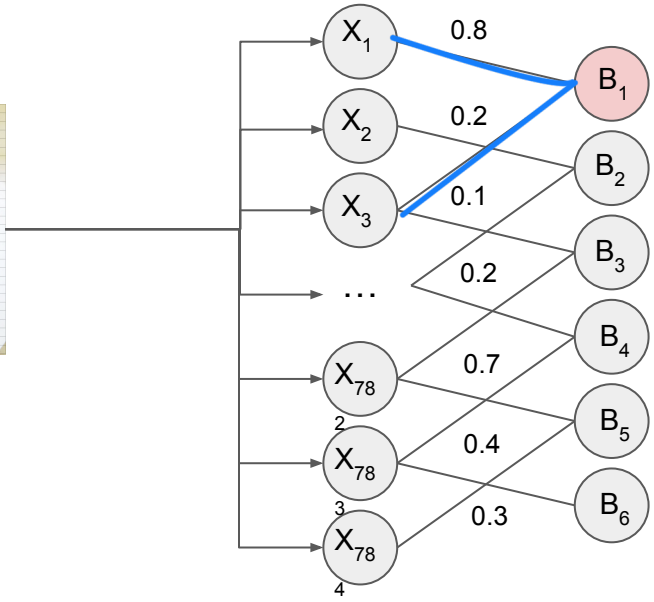


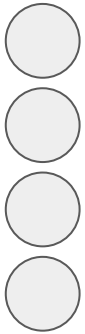
High-Level Concept of Neural Network

$= (x_1 * 0.8 + x_3 * 0.2) + B_1$

Bias associated with the neuron





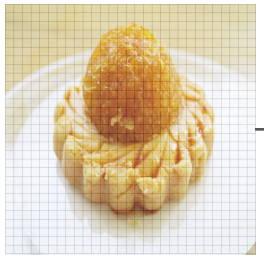


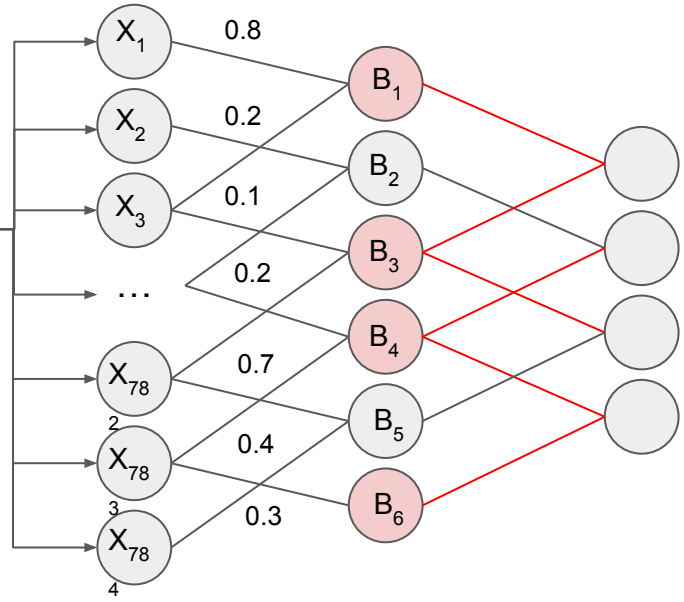
High-Level Concept of Neural Network

$$y = \left[(x_1 * 0.8 + x_3 * 0.2) + B_1 \right]$$

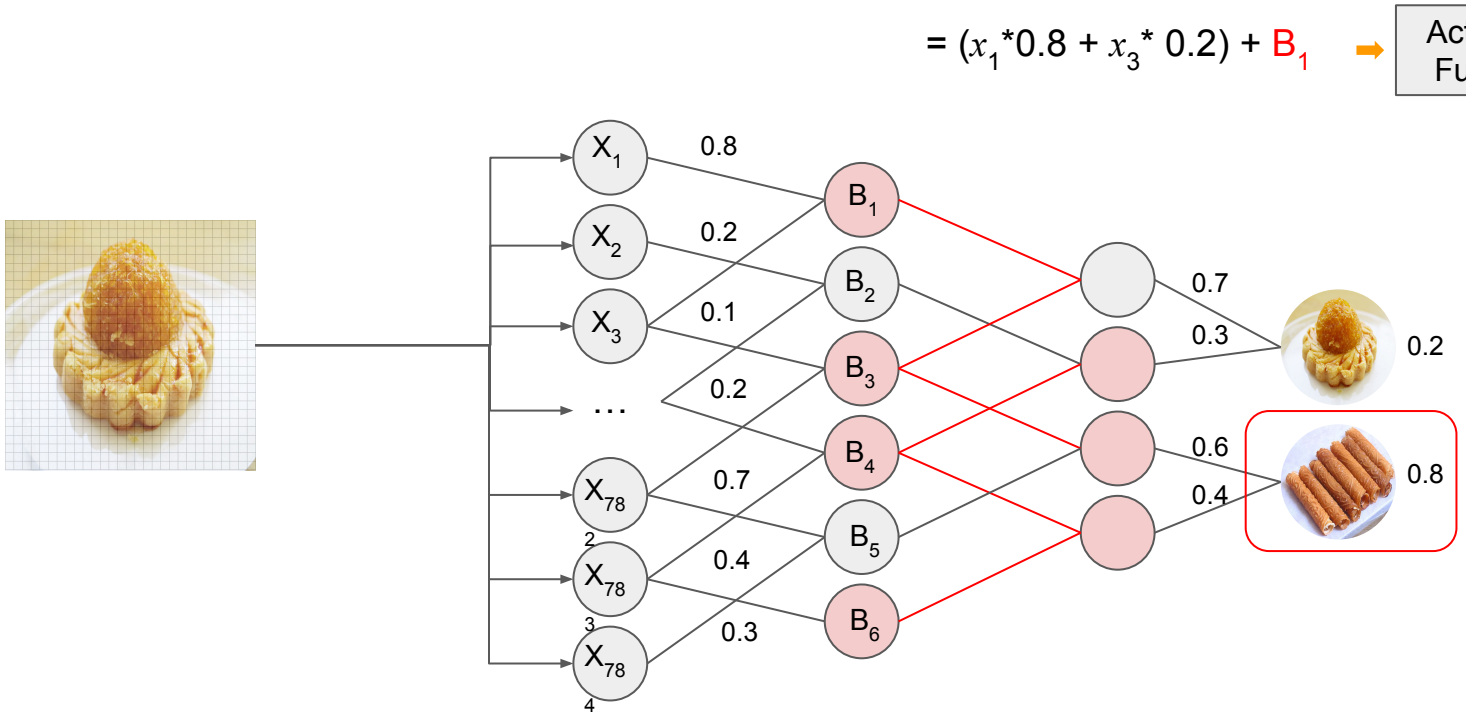
Activation Function

Determine if the neuron will be activated to input for next layer

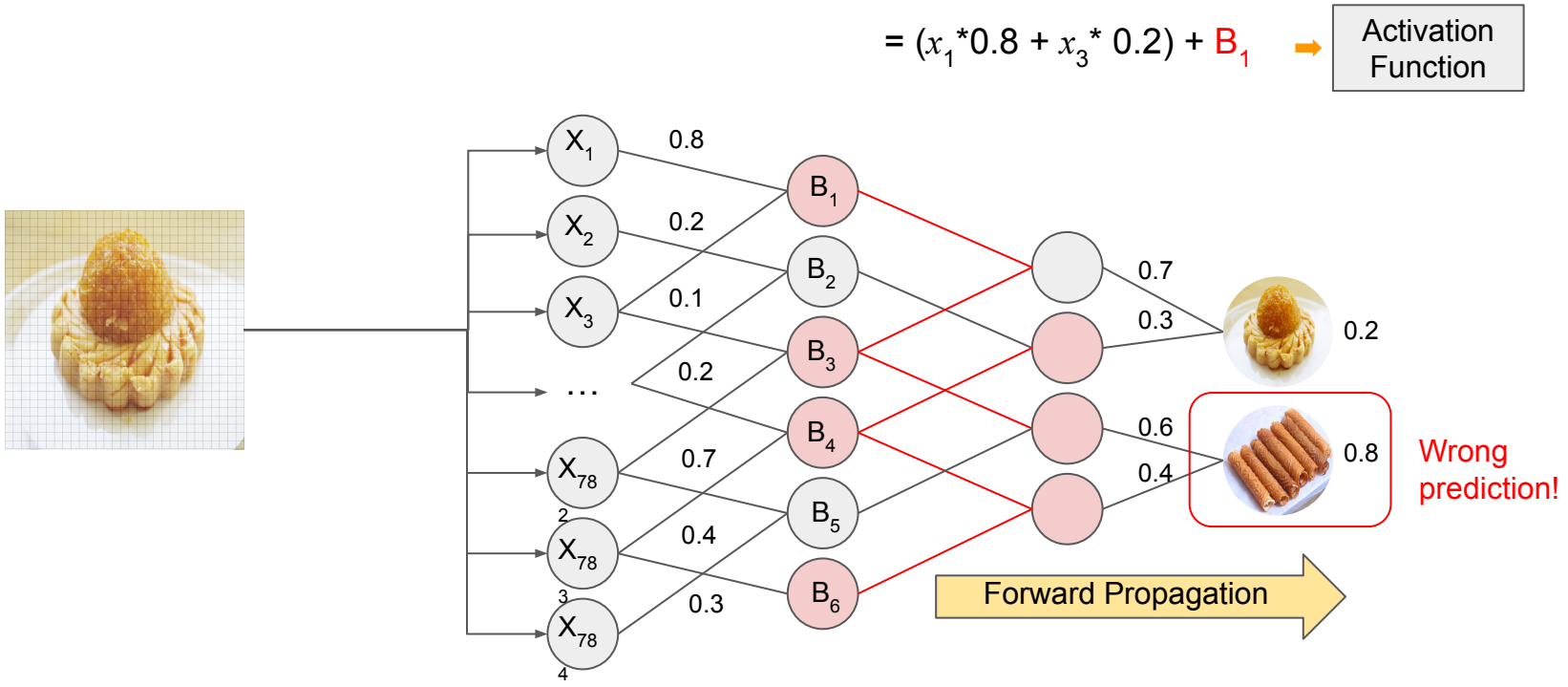




High-Level Concept of Neural Network

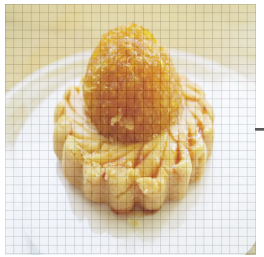


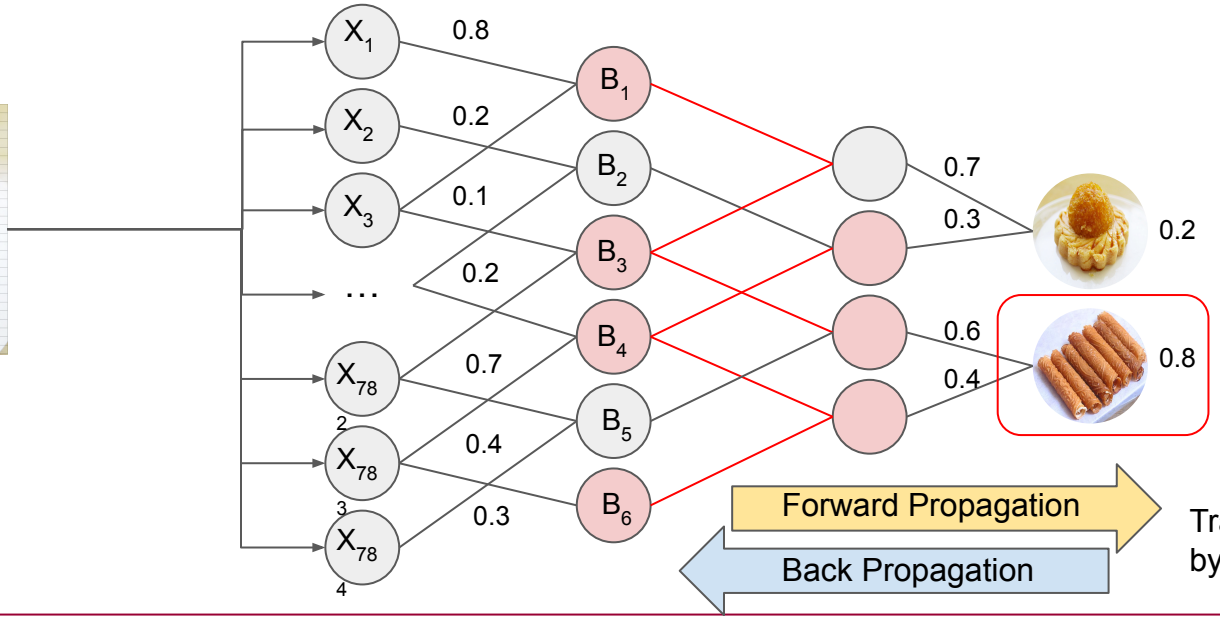
High-Level Concept of Neural Network



High-Level Concept of Neural Network

$= (x_1 * 0.8 + x_3 * 0.2) + B_1$ ➡ Activation Function



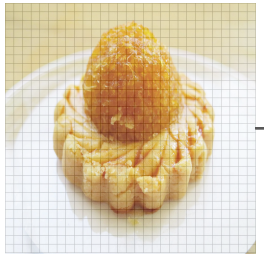


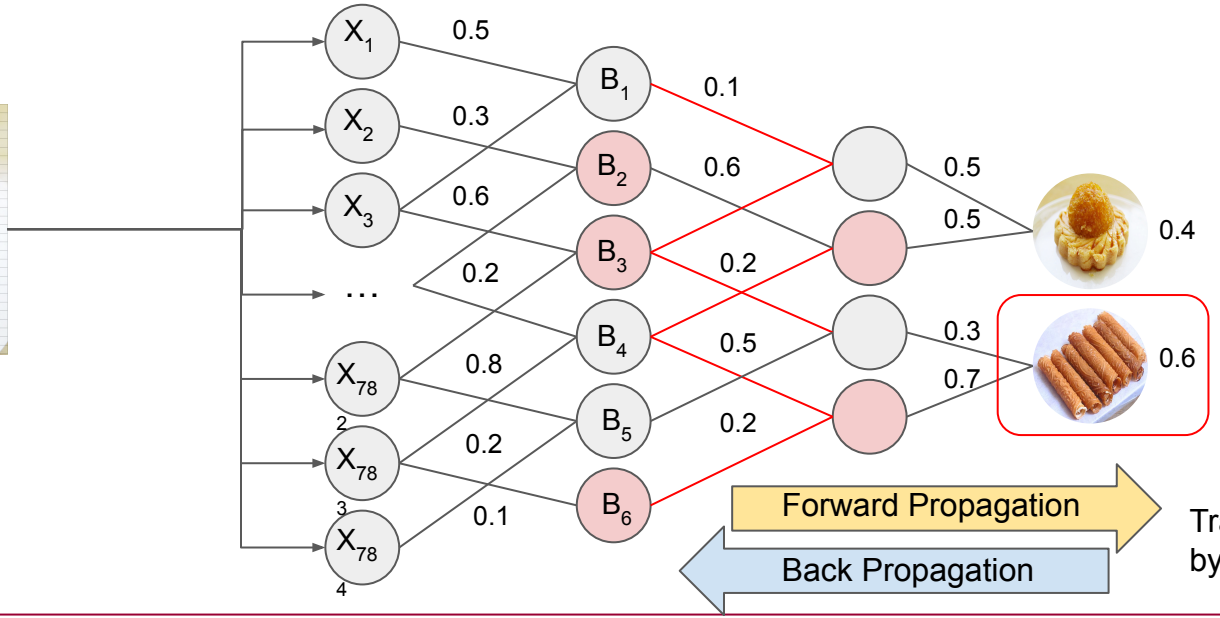
Actual Output	Error
1	+0.8
0	-0.8

Training with back prop by adjusting the weights

High-Level Concept of Neural Network

$= (x_1 * 0.8 + x_3 * 0.2) + B_1$ ➡ Activation Function



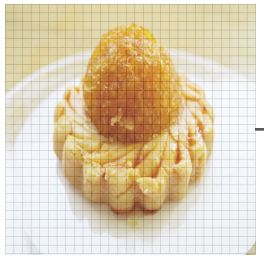


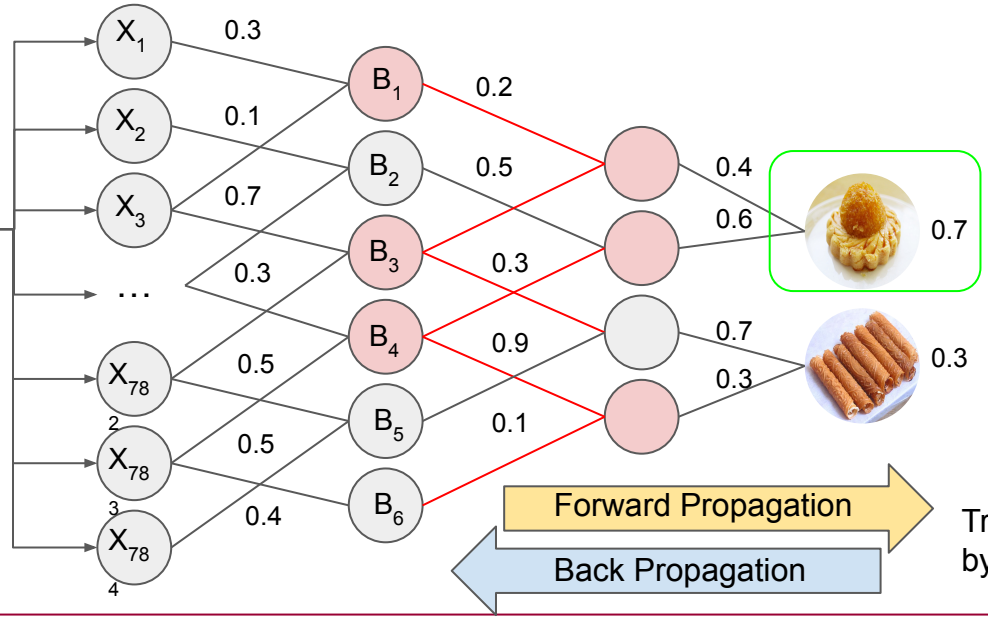
Actual Output	Error
1	+0.6
0	-0.6

Training with back prop by adjusting the weights

High-Level Concept of Neural Network

$= (x_1 * 0.8 + x_3 * 0.2) + B_1$ ➡ Activation Function



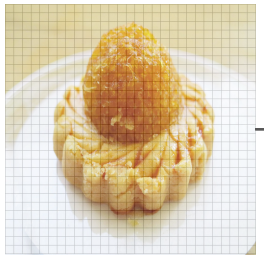


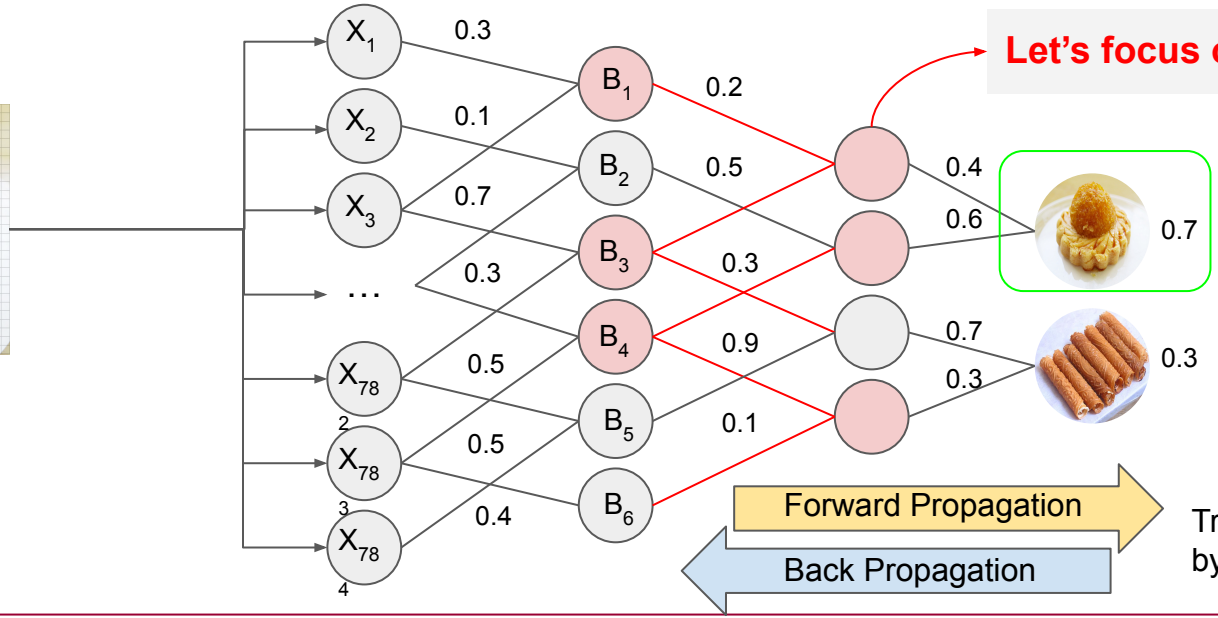
Actual Output	Error
1	+0.3
0	-0.3

Training with back prop by adjusting the weights

High-Level Concept of Neural Network

$= (x_1 * 0.8 + x_3 * 0.2) + B_1$ ➡ Activation Function

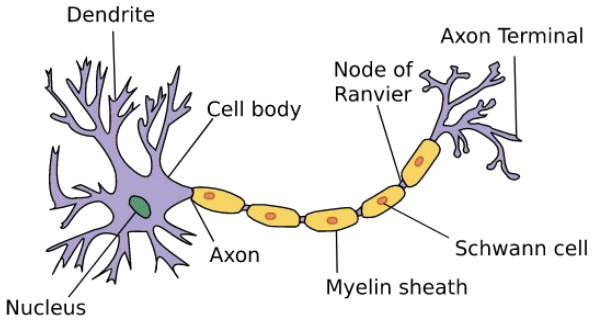




What is Neural Network?

Neurons in the Brain

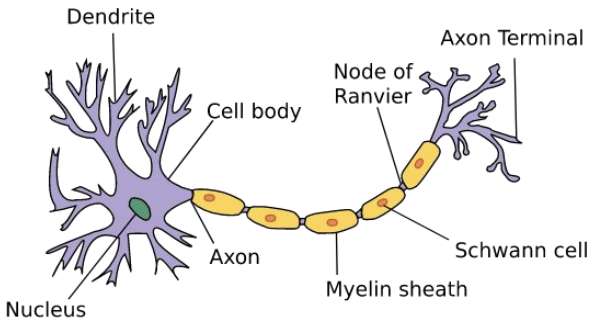
- Neural networks is developed simulating networks of neurons. We will first study how a single neuron looks like.
- Basically neuron is a computational unit that gets a number of inputs, does some computations and send outputs to other neurons through axon.



A single neuron in the brain

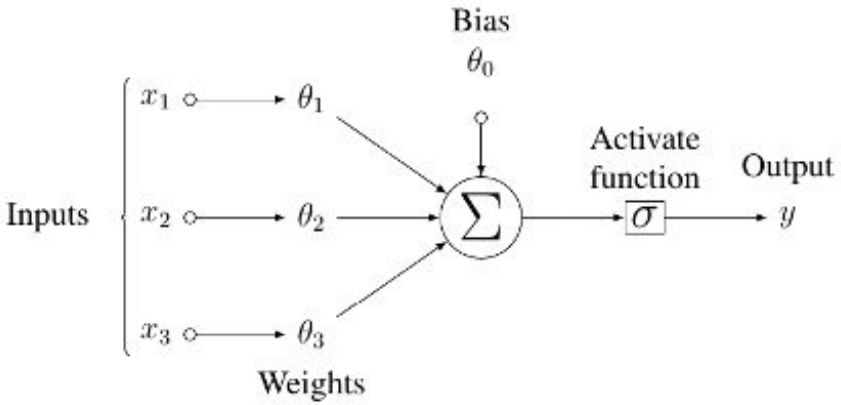
Neurons in the Brain

- Communication between neurons is with little pulses of electricity.
- If the neuron wants to send a message, it sends an electricity through its axon.



A single neuron in the brain

A Single Neuron in Neural Networks



Bias unit, always 1

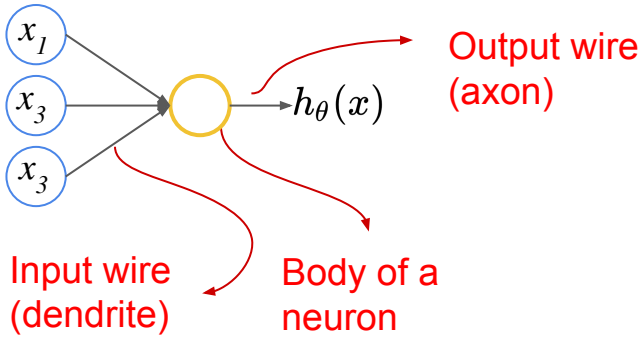
$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

- Each neuron contains three components: inputs, weights (including the bias), activation function.
- Activation function is a *non-linear* function, which is the **key** to the success of neural networks!
- The choice of activation function is *user-specific*, and the design of an optimal activation function remains an open area of research.

A Single Neuron in Neural Networks

Logistic Unit

We will model a neuron as a logistic unit.



$$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$$

$$\theta \epsilon \begin{bmatrix} \theta_0 \\ \theta_1 \\ \dots \\ \theta_n \end{bmatrix}$$

Parameters or weights (NN)

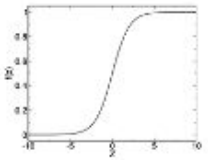
Bias unit, always 1

$$x \epsilon \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix}$$

This is a very simple artificial neuron with sigmoid/logistic activation function

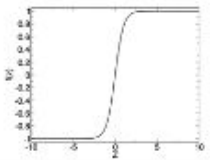
Activation Functions

Sigmoid $f(z) = \frac{1}{1+\exp(-z)}$



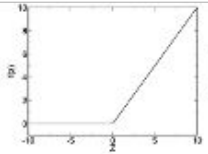
[0,1]

Hyperbolic tangent $f(z) = \tanh(z)$



[-1,1]

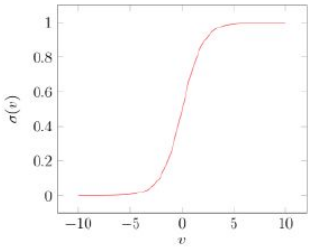
Rectified linear $f(z) = \max(0, z)$



[0,inf]

Logistic/sigmoid function

$$y = \sigma\left(\sum_{i=1}^{d} \theta_i \cdot x_i + \theta_0\right)$$



$$\sigma(x) = \frac{e^x}{1+e^x}$$

Pros:

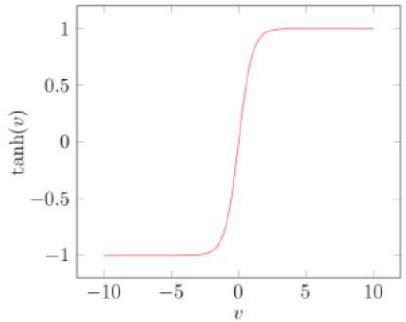
- Smooth gradient, preventing “jumps” in output values.
- Output values bound between 0 and 1, normalizing the output of each neuron.
- Clear predictions—For X above 2 or below -2, tends to bring the Y value (the prediction) to the edge of the curve, very close to 1 or 0. This enables clear predictions.

Cons:

- Vanishing gradient—for very high or very low values of X, there is almost no change to the prediction, causing a vanishing gradient problem. This can result in the network refusing to learn further, or being too slow to reach an accurate prediction.

TanH/ Hyperbolic Tangent function:

$$y = \tanh\left(\sum_{i=1}^{d} \theta_i \cdot x_i + \theta_0\right)$$



$$\tanh(v) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Pros🙂

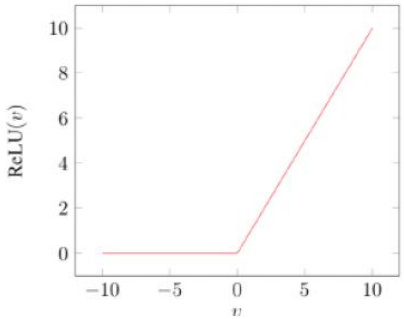
- Zero centered—making it easier to model inputs that have strongly negative, neutral, and strongly positive values. •Otherwise like the Sigmoid function.

Cons😟

- Like the Sigmoid function

ReLU(Rectified Linear Unit) function

$$y = \text{ReLU}(\sum_{i=1}^{d} \theta_i \cdot x_i + \theta_0)$$



$$\text{ReLU}(v) = \max(0, x)$$

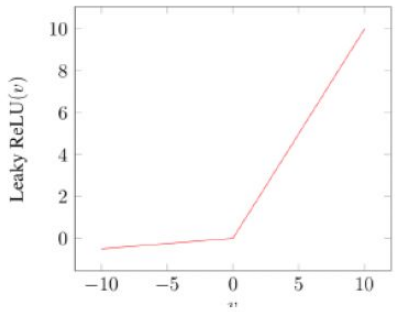
Pros🙂

- Computationally efficient—allows the network to converge very quickly!

Cons😟

- The Dying ReLU problem—when inputs approach zero, or are negative, the gradient of the function becomes zero, the network cannot perform backpropagation and cannot learn.

Leaky ReLU function



Pros🙂

- Prevents dying ReLU problem—this variation of ReLU has a small positive slope in the negative area, so it does enable backpropagation, even for negative input values
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- Otherwise like ReLU

Cons😟

- Results not consistent—leaky ReLU does not provide consistent predictions for negative input values.

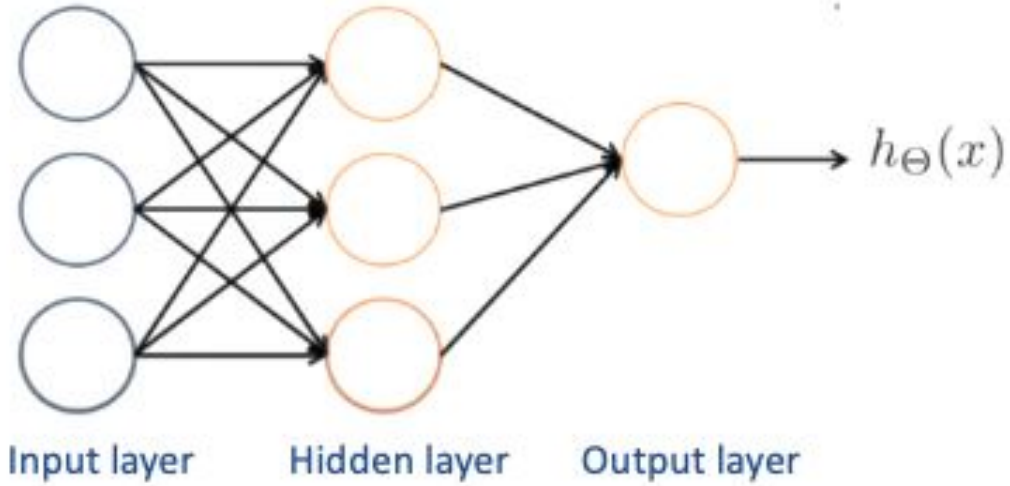
Other activation functions

- Softmax
- Parametric ReLU
- Swish
- Exponential Linear Units (ELU)
- …

What is Neural Network?

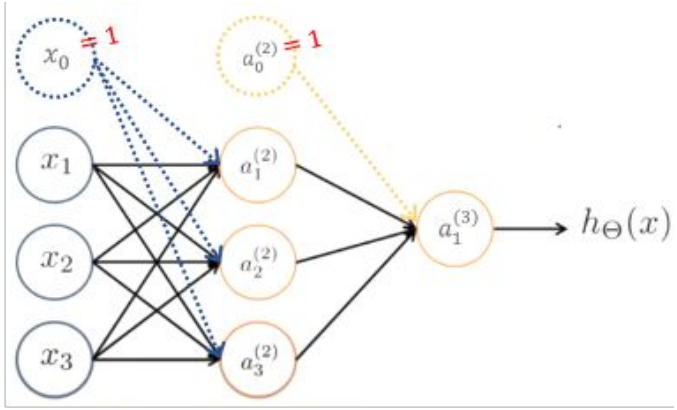
Neural Network

When we connect an ensemble of neurons in a graph is when the magic happens and we get an actual **neural network**.



Neural networks are arranged in **layers**, with **one** input layer (*the first layer*), **one** output layer (*the last layer*) and ***N*** hidden layers in the middle (*N*=1 in this example).

Neural Network - Hypothesis Representation

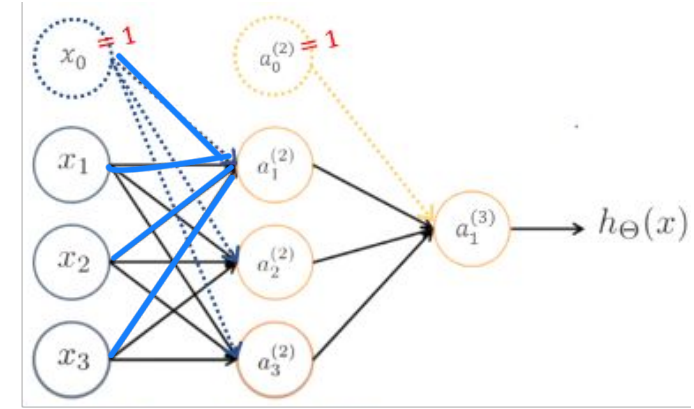


$a_i^{(j)}$: "activation" of unit i in layer j where activation means the value calculated as an output of neuron

$\Theta^{(j)}$: matrix of weights controlling function mapping from layer j to layer $j+1$

Notice that to ease the notation, biases (if applicable) have been incorporated into weight matrices $\Theta^{()}$

Neural Network - Hypothesis Representation



weight from 1st node in 2nd layer

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

three hidden units

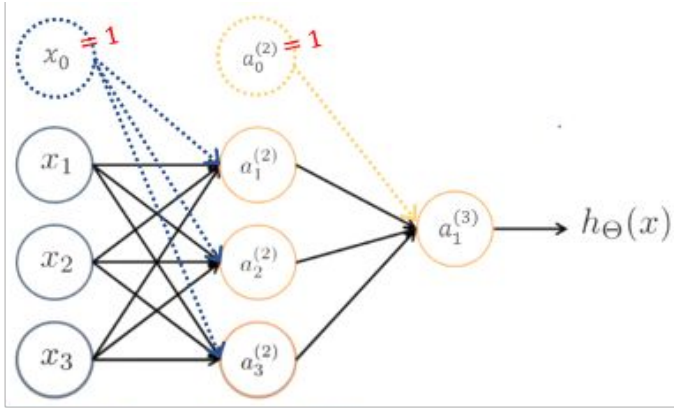
$$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

$\Theta_{10}^{(1)}$: (1) = 1st layer
1 = node 1 in 2nd layer
0 = node 0 in 1st layer

$a_i^{(j)}$: "activation" of unit i in layer j where activation means the value calculated as an output of neuron
$\Theta^{(j)}$: matrix of weights controlling function mapping from layer j to layer $j+1$

Notice that to ease the notation, biases (if applicable) have been incorporated into weight matrices $\Theta^{()}$

Neural Network - Hypothesis Representation



bias

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

three hidden units

$$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

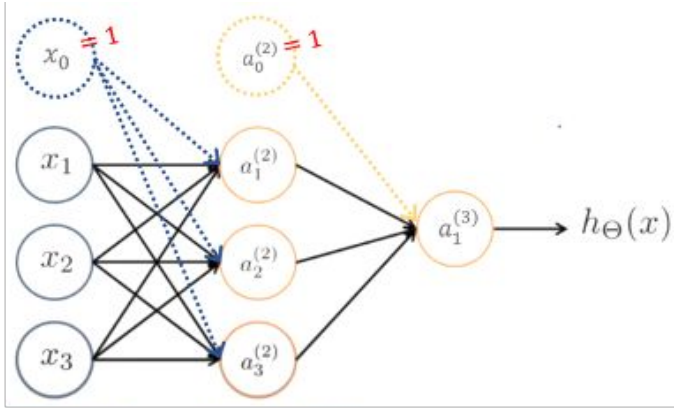
bias

$a_i^{(j)}$: "activation" of unit i in layer j where activation means the value calculated as an output of neuron

$\Theta^{(j)}$: matrix of weights controlling function mapping from layer j to layer $j+1$

Notice that to ease the notation, biases (if applicable) have been incorporated into weight matrices $\Theta^{()}$

Neural Network - Hypothesis Representation



bias

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

three hidden units

$$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

bias

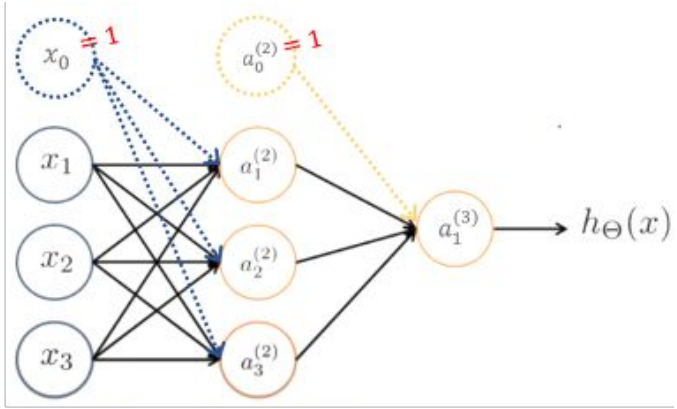
If network has s_j units in layer j, and s_{j+1} units in layer $j+1$, then the dimension of $\Theta^{(j)}$ will be $s_{j+1} \times (s_j + 1)$ when adding bias.

$a_i^{(j)}$: "activation" of unit i in layer j where activation means the value calculated as an output of neuron

$\Theta^{(j)}$: matrix of weights controlling function mapping from layer j to layer $j+1$

Notice that to ease the notation, biases (if applicable) have been incorporated into weight matrices $\Theta^{()}$

Neural Network - Hypothesis Representation



bias

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

three hidden units

$$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

bias

If network has s_j units in layer j, and s_{j+1} units in layer $j+1$, then the dimension of $\Theta^{(j)}$ will be $s_{j+1} \times (s_j + 1)$ when adding bias.

$a_i^{(j)}$: "activation" of unit i in layer j where activation means the value calculated as an output of neuron

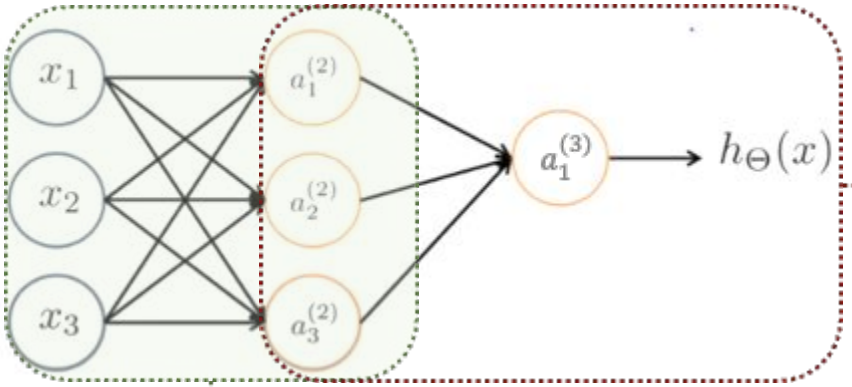
$\Theta^{(j)}$: matrix of weights controlling function mapping from layer j to layer $j+1$

$\Theta^{(1)} \in \mathbb{R}^{3\times4}$

$\Theta^{(2)} \in \mathbb{R}^{1\times4}$

Notice that to ease the notation, biases (if applicable) have been incorporated into weight matrices $\Theta^{()}$

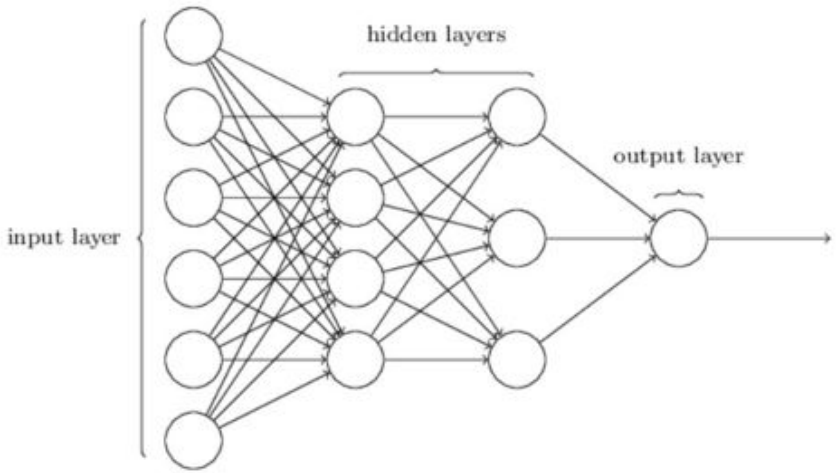
Neural Network



Logistic Regression Unit

NN learns its own features by optimizing the parameters $\Theta^{(1)}$ instead of manually designing polynomial terms to map the data from the original feature space into another feature space (*as we did in non-linear SVM using the kernel trick*).

Multi-layer Neural Network



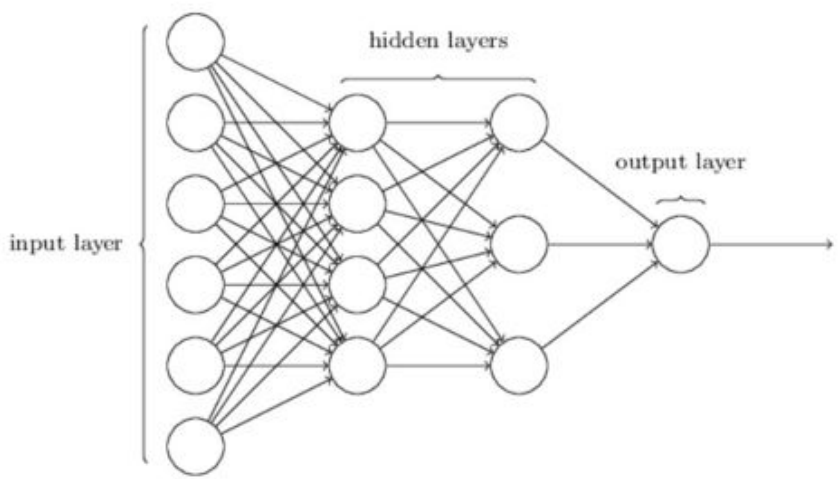
The hidden layers are responsible for learning good representations for the inputs.

To increase the expressiveness of the function that can be possibly learned from the data, we can also consider more hidden layers, resulting in the **multi-layer neural network**, aka multilayer perceptron (MLP).

These neural networks with multiple layers (*e.g.*, MLP, CNN, RNN/LSTM, Transformer) between the input and output layers are referred as **Deep Neural Networks**.

- For example, GPT3 is based on **Transformer** architecture with **175 billion** parameters.
- Scaling Laws for Neural Language Model: **Bigger** is *often* **better**

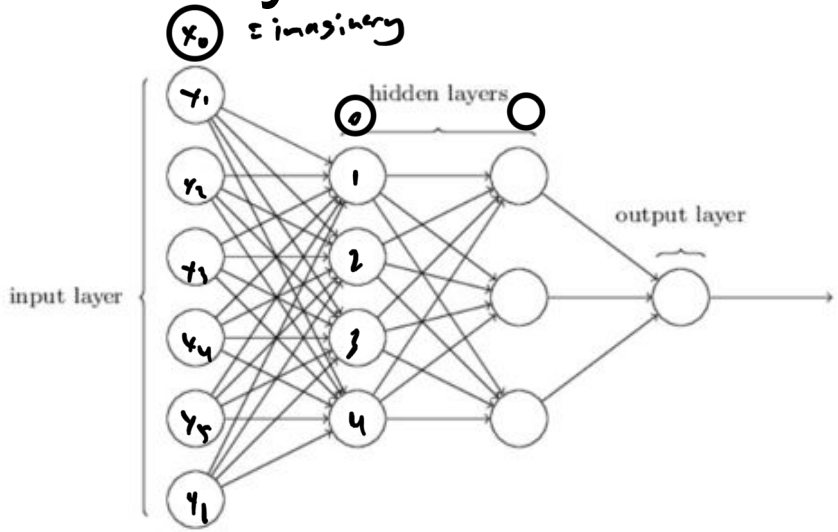
Multi-layer Neural Network



How many learnable parameters does the network depicted here have?

- Assume that the biases are ADDED but not depicted in this figure.

Multi-layer Neural Network

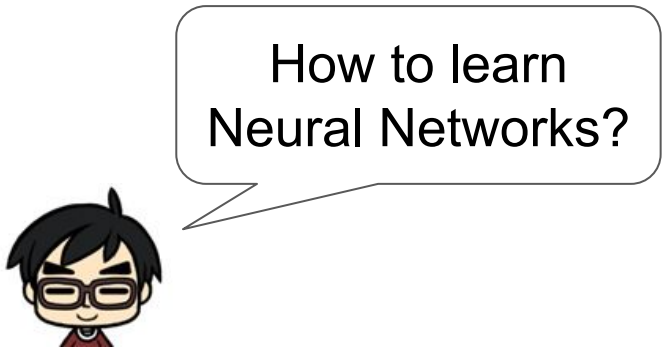


How many learnable parameters does the network depicted here have?

- Assume that the biases are ADDED but not depicted in this figure.

47!

- $\Theta^{(1)} \in \mathbb{R}^{4\times 7}$ are the weights of first layer
- $\Theta^{(2)} \in \mathbb{R}^{3\times 5}$ are the weights of second layer
- $\Theta^{(3)} \in \mathbb{R}^{1\times 4}$ are the weights of third layer


How to learn
Neural Networks?

Learning/Training Procedure

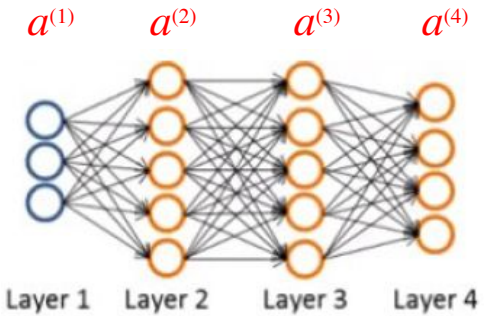
0. Random initialization for weights
1. Forward inputs to get predictions – Forward Propagation
2. Calculate how wrong the predictions are – Cost Function
3. Compute the adjustments (gradients) for weights – Backpropagation
4. Make the adjustment – Optimization method *e.g.* Gradient Descent

Iterate until it converges or meets the maximum iteration steps!

Forward Propagation

- Let's assume we only have **a pair of training data** (x,y).We'll first calculate the forward propagation in order to compute what a hypothesis outputs, given this input.
- $a^{(1)}$ is the activation value of the first layer.

$$
\begin{aligned}
a^{(1)} &= x \\
z^{(2)} &= \Theta^{(1)} a^{(1)} \\
a^{(2)} &= g(z^{(2)}) \\
z^{(3)} &= \Theta^{(2)} a^{(2)} \\
a^{(3)} &= g(z^{(3)}) \\
z^{(4)} &= \Theta^{(3)} a^{(3)} \\
a^{(4)} &= g\left(z^{(4)}\right) = h_\Theta(x)
\end{aligned}
$$



We can compute all the activation values of the neurons in our network.

In order to computer derivatives, we'll use an algorithm called backpropagation.

Cost Function

The cost or loss function measures the quality of our mapping from input to output. This function has a form of this kind:

$$J(\boldsymbol{\Theta}) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}\left(\hat{y}^i, y^i\right) + \lambda\Omega(\boldsymbol{\Theta}), \quad \hat{y}^i = h_{\boldsymbol{\Theta}}\left(x^i\right),$$

where:

- m is the number of training examples, **Θ** is the set of network parameters.
- Ω(·) is the regularization term that uses to mitigate the risk of overfitting (optional) and λ weighs the regularization strength.

The choice of the first term (data loss) depends on the task we want to solve:

- **Classification** – cross-entropy loss (*aka* log loss):

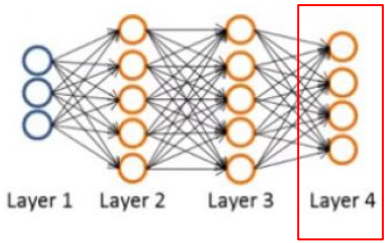
$$\mathcal{L}\left(\hat{y}^i, y^i\right) = -y^i \log \hat{y}^i - (1 - y^i) \log(1 - \hat{y}^i)$$

- **Regression -** Mean Squared Error (MSE):

$$\mathcal{L}\left(\hat{y}^i, y^i\right) = \| y^i - \hat{y}^i \|_2^2$$

Back Propagation

- For each node, we'll compute: $\delta_j^{(l)}$= 'error' of node j in layer l.
- For the 4th layer: $\delta_j^{(4)} = a_j^{(4)} - y_j$ *(the difference between the activation (hypothesis output) and the label)*



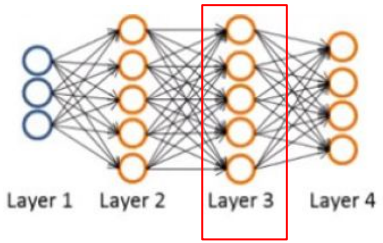
Vectorized implementation:

How to compute the δ terms of earlier layers:

$\delta^{(4)} = a^{(4)} - y$ (dimension is equal to the number of output units in network)

Back Propagation

- For each node, we'll compute: $\delta_j^{(l)}$= 'error' of node j in layer l.
- For the 4th layer: $\delta_j^{(4)} = a_j^{(4)} - y_j$ *(the difference between the activation (hypothesis output) and the label)*



Vectorized implementation:

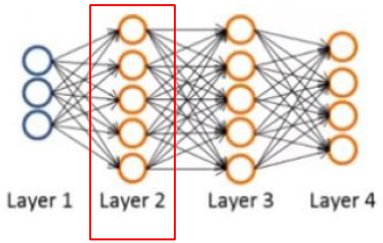
How to compute the δ terms of earlier layers:

$\delta^{(4)} = a^{(4)} - y$ (dimension is equal to the number of output units in network)

$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} .* g'(z^{(3)})$

Back Propagation

- For each node, we'll compute: $\delta_j^{(l)}$= 'error' of node j in layer l.
- For the 4th layer: $\delta_j^{(4)} = a_j^{(4)} - y_j$ *(the difference between the activation (hypothesis output) and the label)*



Vectorized implementation:

How to compute the δ terms of earlier layers:

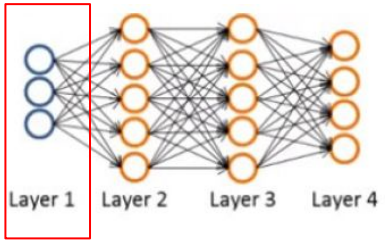
$\delta^{(4)} = a^{(4)} - y$ (dimension is equal to the number of output units in network)

$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} .* g'(z^{(3)})$

$\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} .* g'(z^{(2)})$

Back Propagation

- For each node, we'll compute: $\delta_j^{(l)}$= 'error' of node j in layer l.
- For the 4th layer: $\delta_j^{(4)} = a_j^{(4)} - y_j$ *(the difference between the activation (hypothesis output) and the label)*



Vectorized implementation:

How to compute the δ terms of earlier layers:

$\delta^{(4)} = a^{(4)} - y$ (dimension is equal to the number of output units in network)

$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} .* g'(z^{(3)})$

$\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} .* g'(z^{(2)})$ activation function

What about $\delta^{(1)}$?

Back Propagation

$$\delta^{(4)} = a^{(4)} - y$$
$$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} .* g'(z^{(3)})$$
$$\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} .* g'(z^{(2)})$$

There is no $\delta^{(1)}$ term because the first layer is the input layer, so no error is associated there.

Back propagation term: we start calculating $\delta^{(4)}$, then go back and calculate $\delta^{(3)}$, $\delta^{(2)}$. We are **back propagating** the errors from the output layer to hidden layers.

$$\frac{\partial}{\partial\left(\Theta_{ij}^{(l)}\right)} J(\Theta) = a_j^{(l)} \delta_i^{(l+1)}$$

By performing backpropagation and updating delta terms, you can easily and quickly compute these partial derivatives. :)

Learning/Training Procedure

0. Random initialization for weights
1. Forward Propagation $\hat{y} = \sigma(\boldsymbol{\Theta}_2(\sigma(\boldsymbol{\Theta}_1 \mathbf{x})))$
2. Cost Function $J(\boldsymbol{\Theta}) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^i, y^i) + \lambda\Omega(\boldsymbol{\Theta}), \quad \hat{y}^i = h_{\boldsymbol{\Theta}}(x^i),$
3. Backpropagation
   - Calculate $\frac{\partial J(\boldsymbol{\Theta})}{\partial \hat{y}}$
   - Backward recursion: $\frac{\partial J(\boldsymbol{\Theta})}{\partial \boldsymbol{a}_{i-1}} = \frac{\partial J(\boldsymbol{\Theta})}{\partial \boldsymbol{a}_i}\frac{\partial \boldsymbol{a}_i}{\partial \boldsymbol{a}_{i-1}}; \quad \frac{\partial J(\boldsymbol{\Theta})}{\partial \boldsymbol{\Theta}_i} = \frac{\partial J(\boldsymbol{\Theta})}{\partial \boldsymbol{a}_i}\frac{\partial \boldsymbol{a}_i}{\partial \boldsymbol{\Theta}_i}$
4. Gradient Descent $\boldsymbol{\Theta}_i = \boldsymbol{\Theta}_i - \eta\frac{\partial J(\boldsymbol{\Theta})}{\partial \boldsymbol{\Theta}_i}$

Iterate until it converges or meets the maximum iteration steps!

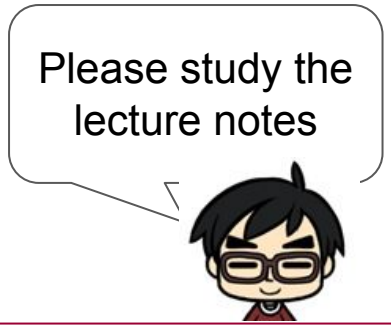
Neural Network Visualization

https://playground.tensorflow.org/

What you should know?

- Why do we need Neural Network?
- What is Neural Network (NN)? Explain the topology of a NN.
- Why can Neural Networks work?
- How to train Neural Network using backpropagation?

Suggestion: If you're not very clear or want to learn more, please do read: "C. Bishop: Pattern Recognition and Machine Learning. Springer, 2006" (recommended text book). It will help you to understand the concept.



Acknowledgement

- *The following material have been referenced or partially used:*
  - *Lecture notes from Prof. Berrak Sisman and Prof. Zhao Na*
  - *National University of Singapore – Pattern Recognition Module (EE5907R)*
  - *National University of Singapore – Neural Networks Module (EE5904R)*
  - *University of Edinburgh, United Kingdom – Machine Learning And Pattern Recognition (MLPR, INFR11130)*
  - *Stanford University – Machine Learning (CS229)*